Table of Contents

Namespace Uralstech.AvLoader

Classes

AvFlow

Class supporting a complete avatar loader flow with fallbacks for each step.

AvFlow.Builder

Fluent builder for AvFlow.

AvMetadata

Information regarding a loaded model.

AvSourceData

Wrapper for a raw container of avatar data.

LoadedAv

A loaded avatar and its associated data.

Interfaces

IAsyncAvPostProcessor

A simple script that can perform an asynchronous post-processing step on an imported avatar.

IAvDataLoader

Interface for an avatar data loader.

IAvImporter

Interface for an avatar importer.

IAvPostProcessor

A simple script that can perform a post-processing step on an imported avatar.

Enums

AvGender

The gender a loaded avatar or of the outfit it is wearing. Can be useful for using specific animation avatars for different types.

AvImageFileExtension

The file extension of an avatar's render(s).

AvModelFileExtension

The file extension of an avatar's model.

AvType

Describes the type of a loaded model.

Class AvFlow

Namespace: Uralstech.AvLoader

Class supporting a complete avatar loader flow with fallbacks for each step.

```
public class AvFlow
```

Inheritance

object ← AvFlow

Constructors

AvFlow(IReadOnlyList<IAvDataLoader>, IReadOnlyList<IAvImporter>)

```
public AvFlow(IReadOnlyList<IAvDataLoader> dataLoaders, IReadOnlyList<IAvImporter>
importers)
```

Parameters

`dataLoaders` IReadOnlyList<IAvDataLoader>

`importers` IReadOnlyList<IAvImporter>

Fields

AsyncPostProcessors

Async post-processors to run after loading the avatar.

```
public IEnumerable<IAsyncAvPostProcessor> AsyncPostProcessors
```

Field Value

IEnumerable<IAsyncAvPostProcessor>

DataLoaders

The loaders to use in the flow, in priority order.

```
public readonly IReadOnlyList<IAvDataLoader> DataLoaders
```

Field Value

IReadOnlyList<IAvDataLoader>

Remarks

Index 0 is used first; if it cannot handle the data, the flow falls back to index 1, then 2, and so on.

Importers

The importers to use in the flow, in priority order.

```
public readonly IReadOnlyList<IAvImporter> Importers
```

Field Value

IReadOnlyList<IAvImporter>

Remarks

Index 0 is used first; if it cannot handle the model format, the flow falls back to index 1, then 2, and so on.

PostProcessors

Post-processors to run after loading the avatar.

```
public IEnumerable<IAvPostProcessor> PostProcessors
```

Field Value

IEnumerable<IAvPostProcessor>

Methods

RunAsync(CancellationToken)

Runs the loader flow completely.

```
public Awaitable<LoadedAv> RunAsync(CancellationToken token = default)
```

Parameters

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar.

TryRunAsync(CancellationToken)

Tries to run the loader flow completely.

```
public Awaitable<LoadedAv?> TryRunAsync(CancellationToken token = default)
```

Parameters

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar if the entire flow completed successfully; null otherwise.

Class AvFlow.Builder

Namespace: Uralstech.AvLoader

Fluent builder for AvFlow.

```
public sealed class AvFlow.Builder
```

Inheritance
object ← AvFlow.Builder

Methods

Build()

Builds the AvFlow.

```
public AvFlow Build()
```

Returns

AvFlow

Remarks

The returned object is independent of the builder and will not reflect subsequent changes made to this builder instance.

Copy()

Creates a copy of the builder.

```
public AvFlow.Builder Copy()
```

Returns

AvFlow.Builder

Remarks

The returned object is independent of the builder and will not reflect subsequent changes made to this builder instance.

From(IAvDataLoader)

Adds an avatar data loader to the flow.

```
public AvFlow.Builder From(IAvDataLoader loader)
```

Parameters

`loader` IAvDataLoader

Returns

AvFlow.Builder

Remarks

Loaders are tried in the order they are added until one succeeds.

From(params IAvDataLoader[])

Adds multiple avatar data loaders to the flow.

```
public AvFlow.Builder From(params IAvDataLoader[] loaders)
```

Parameters

`loaders` IAvDataLoader[]

Returns

AvFlow.Builder

Remarks

Loaders are tried in the order they are added until one succeeds.

ProcessWith(IAvPostProcessor)

Adds a post-processor to the flow.

```
public AvFlow.Builder ProcessWith(IAvPostProcessor postProcessor)
```

Parameters

`postProcessor` IAvPostProcessor

Returns

AvFlow.Builder

Remarks

Note that **all** sync post-processors run *before* **all** async post-processors, regardless of order of addition in the builder.

ProcessWith(params IAvPostProcessor[])

Adds multiple post-processors to the flow.

```
public AvFlow.Builder ProcessWith(params IAvPostProcessor[] postProcessors)
```

Parameters

`postProcessors` IAvPostProcessor[]

Returns

AvFlow.Builder

Remarks

Note that **all** sync post-processors run *before* **all** async post-processors, regardless of order of addition in the builder.

ProcessWithAsync(IAsyncAvPostProcessor)

Adds an asynchronous post-processor to the flow.

```
public AvFlow.Builder ProcessWithAsync(IAsyncAvPostProcessor asyncPostProcessor)
```

Parameters

`asyncPostProcessor` IAsyncAvPostProcessor

Returns

AvFlow.Builder

Remarks

Note that **all** async post-processors run *after* **all** sync post-processors, regardless of order of addition in the builder.

ProcessWithAsync(params IAsyncAvPostProcessor[])

Adds multiple asynchronous post-processors to the flow.

```
public AvFlow.Builder ProcessWithAsync(params IAsyncAvPostProcessor[] asyncPostProcessors)
```

Parameters

`asyncPostProcessors` IAsyncAvPostProcessor[]

Returns

AvFlow.Builder

Remarks

Note that **all** async post-processors run *after* **all** sync post-processors, regardless of order of addition in the builder.

Using(IAvImporter)

Adds an avatar importer to the flow.

```
public AvFlow.Builder Using(IAvImporter importer)
```

Parameters

`importer` IAvImporter

Returns

AvFlow.Builder

Remarks

Importers are evaluated based on format support, then tried in the order they are added until one succeeds.

Using(params IAvImporter[])

Adds multiple avatar importers to the flow.

```
public AvFlow.Builder Using(params IAvImporter[] importers)
```

Parameters

`importers` IAvImporter[]

Returns

AvFlow.Builder

Remarks

Importers are evaluated based on format support, then tried in the order they are added until one succeeds.

Enum AvGender

Namespace: Uralstech.AvLoader

The gender a loaded avatar or of the outfit it is wearing. Can be useful for using specific animation avatars for different types.

```
public enum AvGender
```

Fields

[EnumMember(Value = "feminine")] Feminine = 2

[EnumMember(Value = "masculine")] Masculine = 1

[EnumMember(Value = "neutral")] Neutral = 3

None = 0

Enum AvImageFileExtension

Namespace: Uralstech.AvLoader

The file extension of an avatar's render(s).

```
public enum AvImageFileExtension
```

Fields

`JPEG = 1`

`JPEGAny = 3`

`JPG = 2`

`None = 0`

`PNG = 4`

Class AvMetadata

Namespace: Uralstech.AvLoader

Information regarding a loaded model.

```
public class AvMetadata
```

Inheritance
object ← AvMetadata

Constructors

AvMetadata(string)

```
public AvMetadata(string id)
```

Parameters

`id` string

Fields

AdditionalMetadata

Optional additional metadata for the avatar.

```
public Dictionary<string, string>? AdditionalMetadata
```

Field Value

Dictionary<string, string>

CreatedAt

The creation timestamp of the model.

```
public DateTimeOffset CreatedAt
```

Field Value

DateTimeOffset

Gender

The gender of the model or outfit it is wearing.

```
public AvGender Gender
```

Field Value

AvGender

Id

Unique ID of the avatar.

```
public string Id
```

Field Value

string

SkinTone

The skin tone of the model stored as a hex string (including the #).

```
public string? SkinTone
```

Field Value

string

Type

The type of the model.

```
public AvType Type
```

Field Value

AvType

UpdatedAt

The last time this model was updated.

```
public DateTimeOffset UpdatedAt
```

Field Value

DateTimeOffset

Methods

TryCreateFromBytes(ReadOnlySpan<byte>, out AvMetadata?, Encoding?, bool)

Tries creating a AvMetadata struct from raw bytes.

```
public static bool TryCreateFromBytes(ReadOnlySpan<byte> data, out AvMetadata? metadata,
Encoding? encoding = null, bool throwOnFail = false)
```

Parameters

`data` ReadOnlySpan<byte>

The data to read from.

`metadata` AvMetadata

The decoded struct, null if decoding fails.

`encoding` Encoding

The encoding of the data. Assumes System.Text.Encoding.UTF8 if null.

`throwOnFail` bool

Should the method throw if it couldn't create the AvMetadata?

Returns

bool

true if successful; false otherwise.

TryCreateFromFile(string, out AvMetadata?, bool)

Tries creating a AvMetadata struct from a file.

```
public static bool TryCreateFromFile(string path, out AvMetadata? metadata, bool throwOnFail
= false)
```

Parameters

`path` string

The path to read from.

`metadata` AvMetadata

The decoded struct, null if decoding fails.

`throwOnFail` bool

Should the method throw if it couldn't create the AvMetadata?

Returns

bool

true if successful; false otherwise.

TryCreateFromUriAsync(Uri, CancellationToken?, Encoding?, bool)

Tries creating a AvMetadata struct from data at the given URI.

```
public static Awaitable<AvMetadata?> TryCreateFromUriAsync(Uri uri, CancellationToken? token
= null, Encoding? encoding = null, bool throwOnFail = false)
```

Parameters

`uri` Uri

The URI to load from.

`token` CancellationToken?

`encoding` Encoding

The encoding of the data. Assumes System.Text.Encoding.UTF8 if null.

`throwOnFail` bool

Should the method throw if it couldn't create the AvMetadata?

Returns

Awaitable<AvMetadata>

The decoded AvMetadata struct if successfull; null otherwise.

Enum AvModelFileExtension

Namespace: Uralstech.AvLoader

The file extension of an avatar's model.

```
public enum AvModelFileExtension
```

Fields

GLB = 2

GLTF = 1

GLTFAny = 3

None = 0

VRM = 4

Class AvSourceData

Namespace: Uralstech.AvLoader

Wrapper for a raw container of avatar data.

```
public class AvSourceData
```

Inheritance

object ← AvSourceData

Constructors

AvSourceData(byte[], AvModelFileExtension, string?, AvMetadata, Type, Texture2D?, Texture2D?)

```
public AvSourceData(byte[] model, AvModelFileExtension modelFormat, string? modelPath,
AvMetadata metadata, Type dataLoaderType, Texture2D? fullRender = null, Texture2D?
bustRender = null)
```

Parameters

`model` byte[]

`modelFormat` AvModelFileExtension

`modelPath` string

`metadata` AvMetadata

`dataLoaderType` Type

`fullRender` Texture2D?

`bustRender` Texture2D?

Fields

BustRender

Optional bust render of the avatar.

```
public readonly Texture2D? BustRender
```

Field Value

Texture2D?

DataLoaderType

The type of the data loader which created this.

```
public readonly Type DataLoaderType
```

Field Value

Type

FullRender

Optional full render of the avatar.

```
public readonly Texture2D? FullRender
```

Field Value

Texture2D?

Metadata

Metadata of the avatar.

```
public readonly AvMetadata Metadata
```

Field Value

AvMetadata

Model

The model as a byte[].

```
public readonly byte[] Model
```

Field Value

byte[]

ModelFormat

The file format of the model.

```
public readonly AvModelFileExtension ModelFormat
```

Field Value

AvModelFileExtension

ModelPath

The path/URI to model.

```
public readonly string? ModelPath
```

Field Value

string

Enum AvType

Namespace: Uralstech.AvLoader

Describes the type of a loaded model.

```
public enum AvType
```

Fields

`[EnumMember(Value = "fullbody")] HumanoidFullBody = 1`

A humanoid, full-body model.

`[EnumMember(Value = "fullbody-xr")] HumanoidFullBodyXR = 3`

A humanoid, full-body model for XR use.

`[EnumMember(Value = "halfbody")] HumanoidHalfBody = 2`

A humanoid, half-body model.

`None = 0`

Interface IAsyncAvPostProcessor

Namespace: Uralstech.AvLoader

A simple script that can perform an asynchronous post-processing step on an imported avatar.

```
public interface IAsyncAvPostProcessor
```

Methods

PostProcessAsync(LoadedAv, AvSourceData, CancellationToken)

Runs a post-processing step on the given avatar.

```
Awaitable PostProcessAsync(LoadedAv avatar, AvSourceData rawData, CancellationToken token
= default)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

`token` CancellationToken

Returns

Awaitable

Interface IAvDataLoader

Namespace: Uralstech.AvLoader

Interface for an avatar data loader.

```
public interface IAvDataLoader
```

Remarks

The role of the IAvDataLoader is to fetch the raw binary and JSON data associated with the avatar. They can be chained up to create fallbacks, for example, a FileAvDataLoader could check local caches for existing avatars, then fall back to a URIAvDataLoader to download it from remote sources.

You could also implement your own data loaders to load avatars using APIs provided by cloud services, like Firebase.

Methods

LoadAvatarAsync(bool, CancellationToken)

Tries loading an avatar into an AvSourceData.

```
Awaitable<AvSourceData?> LoadAvatarAsync(bool throwOnFail, CancellationToken token
= default)
```

Parameters

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<AvSourceData>

The loaded data if successful; null on failure.

Interface IAvImporter

Namespace: Uralstech.AvLoader

Interface for an avatar importer.

```
public interface IAvImporter
```

Remarks

The role of the IAvImporter is to parse data returned by an IAvDataLoader and bring the avatar into Unity-space as a GameObject. Like IAvDataLoaders, they can be chained up to create fallbacks, for example, a GLTFastAvImporter for glTF support, falling back to one for .fbx support, etc.

Methods

ImportAvatarAsync(AvSourceData, bool, CancellationToken)

Tries to import the avatar into a scene as a disabled GameObject, along with metadata and any renders.

```
Awaitable<LoadedAv?> ImportAvatarAsync(AvSourceData rawData, bool throwOnFail,
CancellationToken token = default)
```

Parameters

`rawData` AvSourceData

The raw avatar data to process.

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar if successful; null on failure.

Remarks

The returned LoadedAv may contain code to handle requirements of format-supporting plugins. For example, GLTFastAvImporter depends on glTFast, which requires that GltfImport object used to import the avatar be active alongside the avatar's GameObject, and should be disposed after the avatar is no longer needed. Thus, LoadedAv implements System.IDisposable.

SupportsFormat(AvModelFileExtension)

Returns true if this importer can handle the given format; false otherwise.

```
bool SupportsFormat(AvModelFileExtension format)
```

Parameters

`format` AvModelFileExtension

Returns

bool

Interface IAvPostProcessor

Namespace: Uralstech.AvLoader

A simple script that can perform a post-processing step on an imported avatar.

```
public interface IAvPostProcessor
```

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
void PostProcess(LoadedAv avatar, AvSourceData rawData)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

Class LoadedAv

Namespace: Uralstech.AvLoader

A loaded avatar and its associated data.

```
public abstract class LoadedAv
```

Inheritance

object ← LoadedAv

Derived

LoadedGLTFastAv, LoadedUniGLTFAv, LoadedUniVRM10Av

Constructors

LoadedAv(GameObject, AvMetadata, Texture2D?, Texture2D?, Type)

```
protected LoadedAv(GameObject gameObject, AvMetadata metadata, Texture2D? fullRender,
Texture2D? bustRender, Type importerType)
```

Parameters

`gameObject` GameObject

`metadata` AvMetadata

`fullRender` Texture2D?

`bustRender` Texture2D?

`importerType` Type

Fields

BustRender

Optional bust render of the avatar.

```
public readonly Texture2D? BustRender
```

Field Value

Texture2D?

DestroyOnDispose

Deprecated. Use LifetimeObjects and the RegisterLifetimeObject(Object) APIs instead.

```
[Obsolete("Use LifetimeObjects/RegisterLifetimeObject/DeregisterLifetimeObject instead.")]
public readonly List<UnityEngine.Object> DestroyOnDispose
```

Field Value

List<UnityEngine.Object>

FullRender

Optional full render of the avatar.

```
public readonly Texture2D? FullRender
```

Field Value

Texture2D?

GameObject

The avatar's GameObject.

```
public readonly GameObject GameObject
```

Field Value

GameObject

ImporterType

The type of the importer which created this LoadedAv.

```
public readonly Type ImporterType
```

Field Value

Type

Metadata

Metadata of the avatar.

```
public readonly AvMetadata Metadata
```

Field Value

AvMetadata

Properties

LifetimeObjects

Unity objects whose lifetime is owned by this avatar.

```
public IReadOnlyCollection<UnityEngine.Object> LifetimeObjects { get; }
```

Property Value

IReadOnlyCollection<UnityEngine.Object>

Remarks

All registered objects are destroyed when the avatar is disposed. FullRender and BustRender are registered automatically. Call DeregisterLifetimeObject(Object) to take ownership of an object and prevent it from being destroyed.

Methods

DeregisterLifetimeObject(Object)

Removes a Unity object from the avatar's lifetime ownership.

```
public void DeregisterLifetimeObject(UnityEngine.Object unityObject)
```

Parameters

`unityObject` Object

Remarks

After deregistration, the caller becomes responsible for destroying the object. This is can be used to retain FullRender or BustRender.

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

GetAvatarMaterials(bool)

Gets all materials used by the avatar.

```
public IReadOnlyList<Material> GetAvatarMaterials(bool refreshCache = false)
```

Parameters

`refreshCache` bool

Returns

IReadOnlyList<Material>

Remarks

Results are cached. Call with `refreshCache` set to true if renderers or materials change at runtime.

GetAvatarMeshes(bool)

Gets all meshes used by the avatar.

```
public IReadOnlyList<Mesh> GetAvatarMeshes(bool refreshCache = false)
```

Parameters

`refreshCache` bool

Returns

IReadOnlyList<Mesh>

Remarks

Results are cached. Call with `refreshCache` set to true if renderers or meshes change at runtime.

GetAvatarRenderers(bool)

Gets all renderers used by the avatar.

```
public IReadOnlyList<Renderer> GetAvatarRenderers(bool refreshCache = false)
```

Parameters

`refreshCache` bool

Returns

IReadOnlyList<Renderer>

Remarks

Results are cached. Call with `refreshCache` set to true if renderers change at runtime.

GetCapability<T>()

Casts the current avatar object to `T`.

```
public T GetCapability<T>() where T : ICapability
```

Returns

T

The casted result.

Type Parameters

`T`

The capability type to cast to.

ImporterSpecificDispose()

```
protected virtual void ImporterSpecificDispose()
```

RegisterLifetimeObject(Object)

Registers a Unity object to be destroyed when the avatar is disposed.

```
public void RegisterLifetimeObject(UnityEngine.Object unityObject)
```

Parameters

`unityObject` Object

Remarks

Intended for post-processors and extensions that create temporary runtime objects.

ThrowIfDisposed()

```
protected void ThrowIfDisposed()
```

TryGetAvatarMaterials()

Tries to get all materials of the avatar.

```
public virtual IReadOnlyList<Material>? TryGetAvatarMaterials()
```

Returns

IReadOnlyList<Material>

TryGetAvatarMeshes()

Tries to get all meshes of the avatar.

```
public virtual IReadOnlyList<Mesh>? TryGetAvatarMeshes()
```

Returns

IReadOnlyList<Mesh>

TryGetAvatarRenderers()

Tries to get all renderers of the avatar.

```
public virtual IReadOnlyList<Renderer>? TryGetAvatarRenderers()
```

Returns

IReadOnlyList<Renderer>

TryGetCapability<T>(out T?)

Tries to cast the current avatar object to `T`.

```
public bool TryGetCapability<T>(out T? capability) where T : ICapability
```

Parameters

`capability` T

The casted result, or null if casting failed.

Returns

bool

true if casted successfully; false otherwise.

Type Parameters

`T`

The capability type to cast to.

Remarks

Ultimately, this is just syntactic sugar for:

```
if (avatar is T capability)
    ...
else
    ...
```

Namespace Uralstech.AvLoader.Capabilities

Classes

BlendShapeProviderExtensions

Extensions for IBlendShapeProviders.

Interfaces

IAvatarExpressionProvider

Provides semantically meaningful avatar expressions (e.g. Smile, Blink, Angry).

IBlendShapeProvider

Provides access to weighted blend channels.

IBlendShapeProviderBulk

Provides allocation-free bulk access to blendshape channels.

ICapability

Base interface for all avatar capabilities.

IImportedAnimator

Exposes an Animator that is created by the avatar importer.

ILookAtProvider

Provides control over avatar look-at behavior.

IMeshBlendShapeProvider

Provides access to mesh-level blendshapes.

Class BlendShapeProviderExtensions

Namespace: Uralstech.AvLoader.Capabilities

Extensions for IBlendShapeProviders.

```
public static class BlendShapeProviderExtensions
```

Inheritance

object ← BlendShapeProviderExtensions

Methods

GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>)

Retrieves multiple blendshape weights into the provided buffer.

```
public static void GetWeights(this IBlendShapeProvider provider, ReadOnlySpan<string> names,
Span<float> weights)
```

Parameters

`provider` IBlendShapeProvider

`names` ReadOnlySpan<string>

`weights` Span<float>

Remarks

Uses bulk access when available; otherwise falls back to per-channel access. Missing channel names result in zero values.

HasAnyWeight(IBlendShapeProvider, string[], out string?)

Checks if a channel with any one of the given `names` exists in the current provider.

```
public static bool HasAnyWeight(this IBlendShapeProvider current, string[] names, out
string? foundName)
```

Parameters

`current` IBlendShapeProvider

`names` string[]

`foundName` string

The name of the found channel or null if not found.

Returns

bool

true if a channel was found; false otherwise.

SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>)

Sets multiple blendshape weights from a dictionary.

```
public static int SetWeights(this IBlendShapeProvider provider, IReadOnlyDictionary<string,
float> values)
```

Parameters

`provider` IBlendShapeProvider

`values` IReadOnlyDictionary<string, float>

Returns

int

Remarks

This is a convenience overload; missing channel names are ignored.

SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>)

Sets multiple blendshape weights.

```
public static int SetWeights(this IBlendShapeProvider provider, ReadOnlySpan<string> names,
ReadOnlySpan<float> weights)
```

Parameters

`provider` IBlendShapeProvider

`names` ReadOnlySpan<string>

`weights` ReadOnlySpan<float>

Returns

int

The number of weights successfully applied.

Remarks

Uses bulk access when available; otherwise falls back to per-channel access. Missing channel names are ignored.

SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Sets multiple blendshape weights using name–value pairs.

```
public static int SetWeights(this IBlendShapeProvider provider, ReadOnlySpan<(string name,
float weight)> values)
```

Parameters

`provider` IBlendShapeProvider

`values` ReadOnlySpan<(string name, float weight)>

Returns

int

The number of weights successfully applied.

Remarks

Uses bulk access when available; otherwise falls back to per-channel access. Missing channel names are ignored.

Interface IAvatarExpressionProvider

Namespace: Uralstech.AvLoader.Capabilities

Provides semantically meaningful avatar expressions (e.g. Smile, Blink, Angry).

```
public interface IAvatarExpressionProvider : IBlendShapeProvider, ICapability
```

Inherited Members

IBlendShapeProvider.ChannelNames , IBlendShapeProvider.HasWeights , IBlendShapeProvider.GetWeight(string) , IBlendShapeProvider.SetWeight(string, float) , IBlendShapeProvider.HasWeight(string)

Extension Methods

BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Remarks

Expression names are avatar-level concepts and may drive multiple blendshapes, bones, or other mechanisms.

Interface IBlendShapeProvider

Namespace: Uralstech.AvLoader.Capabilities

Provides access to weighted blend channels.

```
public interface IBlendShapeProvider : ICapability
```

Extension Methods
BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) ,
BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) ,
BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) ,
BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) ,
BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Remarks

This interface does not imply any semantic meaning of the channels. Channel names and behavior are implementation-defined.

Properties

ChannelNames

Case-sensitive keys for all available blend weights.

```
IReadOnlyCollection<string> ChannelNames { get; }
```

Property Value

IReadOnlyCollection<string>

HasWeights

Does this avatar have any blendshape weights?

```
bool HasWeights { get; }
```

Property Value

bool

Methods

GetWeight(string)

Gets the weight for the given name.

```
float GetWeight(string name)
```

Parameters

`name` string

Returns

float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this gets the maximum weight.

HasWeight(string)

Checks if a weight channel with the given name exists.

```
bool HasWeight(string name)
```

Parameters

`name` string

Returns

bool

SetWeight(string, float)

Sets the weight for the given name.

```
void SetWeight(string name, float weight)
```

Parameters

`name` string

`weight` float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this sets the weight for all of them.

Interface IBlendShapeProviderBulk

Namespace: Uralstech.AvLoader.Capabilities

Provides allocation-free bulk access to blendshape channels.

```
public interface IBlendShapeProviderBulk : IBlendShapeProvider, ICapability
```

Inherited Members
IBlendShapeProvider.ChannelNames , IBlendShapeProvider.HasWeights , IBlendShapeProvider.GetWeight(string) , IBlendShapeProvider.SetWeight(string, float) , IBlendShapeProvider.HasWeight(string)

Extension Methods
BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Remarks

The semantic meaning of channels (e.g. expressions vs mesh blendshapes) is implementation-defined and should be determined via additional capability interfaces such as IAvatarExpressionProvider or IMeshBlendShapeProvider.

Methods

GetWeights(ReadOnlySpan<string>, Span<float>)

Retrieves the weights of the specified blendshape channels into the provided buffer.

```
void GetWeights(ReadOnlySpan<string> names, Span<float> weights)
```

Parameters

`names` ReadOnlySpan<string>

`weights` Span<float>

Remarks

For channel names that do not exist, the corresponding output value is set to zero.

SetWeights(ReadOnlySpan<string>, ReadOnlySpan<float>)

Sets the weights of the specified blendshape channels.

```
int SetWeights(ReadOnlySpan<string> names, ReadOnlySpan<float> weights)
```

Parameters

`names` ReadOnlySpan<string>

`weights` ReadOnlySpan<float>

Returns

int

The number of weights successfully applied.

Remarks

Channel names that do not exist are ignored; the return value indicates how many weights were applied.

SetWeights(ReadOnlySpan<(string name, float weight)>)

Sets multiple blendshape weights using name–value pairs.

```
int SetWeights(ReadOnlySpan<(string name, float weight)> values)
```

Parameters

`values` ReadOnlySpan<(string name, float weight)>

Returns

int

The number of weights successfully applied.

Remarks

Channel names that do not exist are ignored; the return value indicates how many weights were applied.

Interface ICapability

Namespace: Uralstech.AvLoader.Capabilities

Base interface for all avatar capabilities.

```
public interface ICapability
```

Interface IImportedAnimator

Namespace: Uralstech.AvLoader.Capabilities

Exposes an Animator that is created by the avatar importer.

```
public interface IImportedAnimator : ICapability
```

Remarks

This capability represents importer- or runtime-library-defined animation support. For example, UniVRM generates an UnityEngine.Animator as part of the import process.

Animators added later by post-processors or user code are not considered part of this capability and should be accessed directly from GameObject.

Properties

Animator

The importer-generated animator.

```
Animator Animator { get; }
```

Property Value

Animator

Interface ILookAtProvider

Namespace: Uralstech.AvLoader.Capabilities

Provides control over avatar look-at behavior.

```
public interface ILookAtProvider : ICapability
```

Remarks

The implementation of look-at (e.g. head, eyes, constraints, or animation) is implementation-defined.

Methods

ClearTarget()

Clears the look-at target.

```
void ClearTarget()
```

Remarks

Implementations should reset the avatar to its default gaze.

SetTarget(Transform)

Sets a transform for the avatar to look at.

```
void SetTarget(Transform transform)
```

Parameters

`transform` Transform

Remarks

Implementations should track the transform over time.

SetTarget(Vector3)

Sets a fixed world-space position for the avatar to look at.

```
void SetTarget(Vector3 worldPosition)
```

Parameters

`worldPosition` Vector3

Interface IMeshBlendShapeProvider

Namespace: Uralstech.AvLoader.Capabilities

Provides access to mesh-level blendshapes.

```
public interface IMeshBlendShapeProvider : IBlendShapeProvider, ICapability
```

Inherited Members

IBlendShapeProvider.ChannelNames , IBlendShapeProvider.HasWeights , IBlendShapeProvider.GetWeight(string) , IBlendShapeProvider.SetWeight(string, float) , IBlendShapeProvider.HasWeight(string)

Extension Methods

BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Remarks

Names correspond to mesh blendshape channels and are not guaranteed to represent avatar expressions.

Namespace Uralstech.AvLoader.DataLoaders

Classes

FileAvDataLoader

Loads avatars from on-device files.

URIAvDataLoader

Loads avatar data from remote URIs via UnityWebRequest.Get(Uri). Downloads are performed in parallel.

Class FileAvDataLoader

Namespace: Uralstech.AvLoader.DataLoaders

Loads avatars from on-device files.

```
public class FileAvDataLoader : IAvDataLoader
```

Inheritance

object ← FileAvDataLoader

Implements

IAvDataLoader

Constructors

FileAvDataLoader(string, string, string?, string?)

Creates a loader configured to load avatar data from paths to each file.

```
public FileAvDataLoader(string modelFilePath, string metadataFilePath, string?
fullRenderFilePath = null, string? bustRenderFilePath = null)
```

Parameters

`modelFilePath` string

Path to the model.

`metadataFilePath` string

Path to the avatar metadata.

`fullRenderFilePath` string

Path to the full render of the avatar (optional).

`bustRenderFilePath` string

Path to the bust render of the avatar (optional).

Exceptions

NotSupportedException

Thrown if the file extension of `modelFilePath`, `fullRenderFilePath` or `bustRenderFilePath` is not supported.

FileAvDataLoader(string, AvModelFileExtension, AvImageFileExtension, AvImageFileExtension, bool, bool)

Creates a loader configured to load avatar data from the directory at `basePath`.

```
public FileAvDataLoader(string basePath, AvModelFileExtension modelFormat =
AvModelFileExtension.None, AvImageFileExtension fullRenderFormat =
AvImageFileExtension.None, AvImageFileExtension bustRenderFormat =
AvImageFileExtension.None, bool shouldLoadFullRender = false, bool shouldLoadBustRender
= false)
```

Parameters

`basePath` string

The base directory containing the avatar and its metadata.

`modelFormat` AvModelFileExtension

The expected file extension(s) of the model, leave as None to auto detect of all supported formats.

`fullRenderFormat` AvImageFileExtension

The expected file extension(s) of the model's full render, leave as None to auto detect of all supported formats.

`bustRenderFormat` AvImageFileExtension

The expected file extension(s) of the model's bust render, leave as None to auto detect of all supported formats.

`shouldLoadFullRender` bool

Should the full render be loaded?

`shouldLoadBustRender` bool

Should the bust render be loaded?

Remarks

`basePath` **must** contain the avatar and associated data in the following structure:

```
basePath/
├─ model.[supported model extension]      # The avatar's model
├─ metadata.json                          # Metadata
├─ full.[supported image extension]       # Full render of the avatar (optional)
└─ bust.[supported image extension]       # Bust render of the avatar (optional)
```

If you want to use a different file structure, call FileAvDataLoader(string, string, string?, string?) instead.

Methods

LoadAvatarAsync(bool, CancellationToken)

Tries loading an avatar into an AvSourceData.

```
public Awaitable<AvSourceData?> LoadAvatarAsync(bool throwOnFail, CancellationToken token
= default)
```

Parameters

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<AvSourceData>

The loaded data if successful; null on failure.

TrySearchForBustRender(string, AvImageFileExtension, out string?, out AvImageFileExtension)

Searches for a supported bust-render image file inside `directory` using the default bust-render file name.

```
public static bool TrySearchForBustRender(string directory, AvImageFileExtension
searchFilter, out string? foundPath, out AvImageFileExtension foundExtension)
```

Parameters

`directory` string

The directory to search in.

`searchFilter` AvImageFileExtension

The expected image file extension(s). Use None or grouped values to search across all supported formats.

`foundPath` string

When this method returns true, contains the full path to the found image file.

`foundExtension` AvImageFileExtension

When this method returns true, contains the detected image file extension.

Returns

bool

true if a valid image file was found; otherwise, false.

TrySearchForFullRender(string, AvImageFileExtension, out string?, out AvImageFileExtension)

Searches for a supported full-render image file inside `directory` using the default full-render file name.

```
public static bool TrySearchForFullRender(string directory, AvImageFileExtension
searchFilter, out string? foundPath, out AvImageFileExtension foundExtension)
```

Parameters

`directory` string

The directory to search in.

`searchFilter` AvImageFileExtension

The expected image file extension(s). Use None or grouped values to search across all supported formats.

`foundPath` string

When this method returns true, contains the full path to the found image file.

`foundExtension` AvImageFileExtension

When this method returns true, contains the detected image file extension.

Returns

bool

true if a valid image file was found; otherwise, false.

TrySearchForImageFile(string, string, AvImageFileExtension, out string?, out AvImageFileExtension)

Searches for a supported image file inside `directory` using a custom base file name.

```
public static bool TrySearchForImageFile(string directory, string
fileNameWithoutExtensino, AvImageFileExtension searchFilter, out string? foundPath, out
AvImageFileExtension foundExtension)
```

Parameters

`directory` string

The directory to search in.

`fileNameWithoutExtensino` string

The image file name without an extension.

`searchFilter` AvImageFileExtension

The expected image file extension(s). Use None or grouped values to search across all supported formats.

`foundPath` string

When this method returns true, contains the full path to the found image file.

`foundExtension` AvImageFileExtension

When this method returns true, contains the detected image file extension.

Returns

bool

true if a valid image file was found; otherwise, false.

TrySearchForModelFile(string, string, AvModelFileExtension, out string?, out AvModelFileExtension)

Searches for a supported avatar model file inside `directory` using a custom base file name.

```
public static bool TrySearchForModelFile(string directory, string
fileNameWithoutExtensino, AvModelFileExtension searchFilter, out string? foundPath, out
AvModelFileExtension foundExtension)
```

Parameters

`directory` string

The directory to search in.

`fileNameWithoutExtensino` string

The model file name without an extension.

`searchFilter` AvModelFileExtension

The expected model file extension(s). Use None or grouped values to search across all supported formats.

`foundPath` string

When this method returns true, contains the full path to the found model file.

`foundExtension` AvModelFileExtension

When this method returns true, contains the detected model file extension.

Returns

bool

true if a valid model file was found; otherwise, false.

TrySearchForModelFile(string, AvModelFileExtension, out string?, out AvModelFileExtension)

Searches for a supported avatar model file inside `directory` using the default model file name.

```
public static bool TrySearchForModelFile(string directory, AvModelFileExtension
searchFilter, out string? foundPath, out AvModelFileExtension foundExtension)
```

Parameters

`directory` string

The directory to search in.

`searchFilter` AvModelFileExtension

The expected model file extension(s). Use None or grouped values to search across all supported formats.

`foundPath` string

When this method returns true, contains the full path to the found model file.

`foundExtension` AvModelFileExtension

When this method returns true, contains the detected model file extension.

Returns

bool

true if a valid model file was found; otherwise, false.

Class URIAvDataLoader

Namespace: Uralstech.AvLoader.DataLoaders

Loads avatar data from remote URIs via UnityWebRequest.Get(Uri). Downloads are performed in parallel.

```
public class URIAvDataLoader : IAvDataLoader
```

Inheritance

object ← URIAvDataLoader

Implements

IAvDataLoader

Constructors

URIAvDataLoader(string, string, AvModelFileExtension, string?, string?, Encoding?)

Creates a loader configured to download avatar data from explicit URIs.

```
public URIAvDataLoader(string modelURI, string metadataURI, AvModelFileExtension
modelFormat, string? fullRenderURI = null, string? bustRenderURI = null, Encoding?
metadataEncoding = null)
```

Parameters

`modelURI` string

Direct URI to the avatar model file.

`metadataURI` string

Direct URI to the metadata JSON file.

`modelFormat` AvModelFileExtension

The file format/extension of the model. Must be explicitly specified (no auto-detection like with FileAvDataLoader).

`fullRenderURI` string

Optional URI to the full-body render image.

`bustRenderURI` string

Optional URI to the bust/half-body render image.

`metadataEncoding` Encoding

Text encoding for the metadata JSON. Defaults to UTF-8.

Exceptions

ArgumentException

Thrown if `modelFormat` is None, as URI-based loading requires an explicit format.

URIAvDataLoader(Uri, Uri, AvModelFileExtension, Uri?, Uri?, Encoding?)

Creates a loader configured to download avatar data from explicit URIs.

```
public URIAvDataLoader(Uri modelURI, Uri metadataURI, AvModelFileExtension modelFormat, Uri?
fullRenderURI = null, Uri? bustRenderURI = null, Encoding? metadataEncoding = null)
```

Parameters

`modelURI` Uri

Direct URI to the avatar model file.

`metadataURI` Uri

Direct URI to the metadata JSON file.

`modelFormat` AvModelFileExtension

The file format/extension of the model. Must be explicitly specified (no auto-detection like with File AvDataLoader).

`fullRenderURI` Uri

Optional URI to the full-body render image.

`bustRenderURI` Uri

Optional URI to the bust/half-body render image.

`metadataEncoding` Encoding

Text encoding for the metadata JSON. Defaults to UTF-8.

Exceptions

ArgumentException

Thrown if `modelFormat` is None, as URI-based loading requires an explicit format.

Fields

WebRequestConfigurationCallback

Called each time a new UnityWebRequest is created, e.g. for auth configuration.

```
public Func<UnityWebRequest, Awaitable>? WebRequestConfigurationCallback
```

Field Value

Func<UnityWebRequest, Awaitable>

Methods

LoadAvatarAsync(bool, CancellationToken)

Tries loading an avatar into an AvSourceData.

```
public Awaitable<AvSourceData?> LoadAvatarAsync(bool throwOnFail, CancellationToken token
= default)
```

Parameters

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<AvSourceData>

The loaded data if successful; null on failure.

Namespace Uralstech.AvLoader.Importers

Classes

GLTFastAvImporter

Imports glTF avatars using glTFast.

LoadedGLTFastAv

A loaded glTFast glTF avatar.

LoadedUniGLTFAv

A loaded UniGLTF glTF avatar.

LoadedUniVRM10Av

A loaded UniVRM VRM10 avatar.

UniGLTFAvImporter

Imports glTF avatars using UniGLTF.

UniVRM10AvImporter

Imports VRM avatars using UniVRM10.

Class GLTFastAvImporter

Namespace: Uralstech.AvLoader.Importers

Imports glTF avatars using glTFast.

```
public class GLTFastAvImporter : IAvImporter
```

Inheritance

object ← GLTFastAvImporter

Implements

IAvImporter

Fields

DeferAgent

Optional custom glTFast defer agent.

```
public IDeferAgent? DeferAgent
```

Field Value

IDeferAgent?

DownloadProvider

Optional custom glTFast download provider.

```
public IDownloadProvider? DownloadProvider
```

Field Value

IDownloadProvider?

ImportSettings

Optional custom glTFast ImportSettings.

```
public ImportSettings? ImportSettings
```

Field Value

ImportSettings?

InstantiationSettings

Optional custom glTFast InstantiationSettings.

```
public InstantiationSettings? InstantiationSettings
```

Field Value

InstantiationSettings?

InstantiatorFactory

Optional factory for custom glTFast GameObjectInstantiators.

```
public Func<GltfImport, GameObject, Awaitable<GameObjectInstantiator>>? InstantiatorFactory
```

Field Value

Func<GltfImport, GameObject, Awaitable<GameObjectInstantiator>>

Logger

Optional custom glTFast logger.

```
public ICodeLogger? Logger
```

Field Value

ICodeLogger?

MaterialGenerator

Optional custom glTFast material generator.

```
public IMaterialGenerator? MaterialGenerator
```

Field Value

IMaterialGenerator?

Methods

ImportAvatarAsync(AvSourceData, bool, CancellationToken)

Tries to import the avatar into a scene as a disabled GameObject, along with metadata and any renders.

```
public Awaitable<LoadedAv?> ImportAvatarAsync(AvSourceData rawData, bool throwOnFail,
CancellationToken token = default)
```

Parameters

`rawData` AvSourceData

The raw avatar data to process.

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar if successful; null on failure.

Remarks

The returned LoadedAv may contain code to handle requirements of format-supporting plugins. For example, GLTFastAvImporter depends on glTFast, which requires that GltfImport object used to import the avatar be active alongside the avatar's GameObject, and should be disposed after the avatar is no longer needed. Thus, LoadedAv implements System.IDisposable.

SupportsFormat(AvModelFileExtension)

Returns true if this importer can handle the given format; false otherwise.

```
public bool SupportsFormat(AvModelFileExtension format)
```

Parameters

`format` AvModelFileExtension

Returns

bool

Class LoadedGLTFastAv

Namespace: Uralstech.AvLoader.Importers

A loaded glTFast glTF avatar.

```
public class LoadedGLTFastAv : LoadedAv, IMeshBlendShapeProvider,
IBlendShapeProvider, ICapability
```

Inheritance

object ← LoadedAv ← LoadedGLTFastAv

Implements

IMeshBlendShapeProvider, IBlendShapeProvider, ICapability

Inherited Members

LoadedAv.GameObject , LoadedAv.Metadata , LoadedAv.FullRender , LoadedAv.BustRender , LoadedAv.DestroyOnDispose , LoadedAv.LifetimeObjects , LoadedAv.ImporterType , LoadedAv.RegisterLifetimeObject(UnityEngine.Object) , LoadedAv.DeregisterLifetimeObject(UnityEngine.Object) , LoadedAv.TryGetCapability<T>(out T) , LoadedAv.GetCapability<T>() , LoadedAv.GetAvatarMaterials(bool) , LoadedAv.GetAvatarRenderers(bool) , LoadedAv.GetAvatarMeshes(bool) , LoadedAv.TryGetAvatarRenderers() , LoadedAv.ThrowIfDisposed() , LoadedAv.Dispose()

Extension Methods

BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Constructors

LoadedGLTFastAv(GameObject, GltfImport, AvMetadata, Texture2D?, Texture2D?, Type)

```
public LoadedGLTFastAv(GameObject gameObject, GltfImport import, AvMetadata metadata,
Texture2D? fullRender, Texture2D? bustRender, Type importerType)
```

Parameters

`gameObject` GameObject

`import` GltfImport

`metadata` AvMetadata

`fullRender` Texture2D?

`bustRender` Texture2D?

`importerType` Type

Fields

Import

The glTFast import associated with the avatar.

```
public readonly GltfImport Import
```

Field Value

GltfImport

Properties

ChannelNames

Case-sensitive keys for all available blend weights.

```
public IReadOnlyCollection<string> ChannelNames { get; }
```

Property Value

IReadOnlyCollection<string>

HasWeights

Does this avatar have any blendshape weights?

```
public bool HasWeights { get; }
```

Property Value

bool

Methods

GetWeight(string)

Gets the weight for the given name.

```
public float GetWeight(string name)
```

Parameters

`name` string

Returns

float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this gets the maximum weight.

HasWeight(string)

Checks if a weight channel with the given name exists.

```
public bool HasWeight(string name)
```

Parameters

`name` string

Returns

bool

ImporterSpecificDispose()

```
protected override void ImporterSpecificDispose()
```

SetWeight(string, float)

Sets the weight for the given name.

```
public void SetWeight(string name, float weight)
```

Parameters

`name` string

`weight` float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this sets the weight for all of them.

TryGetAvatarMaterials()

Tries to get all materials of the avatar.

```
public override IReadOnlyList<Material>? TryGetAvatarMaterials()
```

Returns

IReadOnlyList<Material>

TryGetAvatarMeshes()

Tries to get all meshes of the avatar.

```
public override IReadOnlyList<Mesh>? TryGetAvatarMeshes()
```

Returns

IReadOnlyList<Mesh>

Class LoadedUniGLTFAv

Namespace: Uralstech.AvLoader.Importers

A loaded UniGLTF glTF avatar.

```
public class LoadedUniGLTFAv : LoadedAv, IMeshBlendShapeProvider,
IBlendShapeProvider, ICapability
```

Inheritance

object ← LoadedAv ← LoadedUniGLTFAv

Implements

IMeshBlendShapeProvider, IBlendShapeProvider, ICapability

Inherited Members

LoadedAv.GameObject , LoadedAv.Metadata , LoadedAv.FullRender , LoadedAv.BustRender , LoadedAv.DestroyOnDispose , LoadedAv.LifetimeObjects , LoadedAv.ImporterType , LoadedAv.RegisterLifetimeObject(UnityEngine.Object) , LoadedAv.DeregisterLifetimeObject(UnityEngine.Object) , LoadedAv.TryGetCapability<T>(out T) , LoadedAv.GetCapability<T>() , LoadedAv.GetAvatarMaterials(bool) , LoadedAv.GetAvatarRenderers(bool) , LoadedAv.GetAvatarMeshes(bool) , LoadedAv.ThrowIfDisposed() , LoadedAv.Dispose()

Extension Methods

BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Constructors

LoadedUniGLTFAv(GameObject, RuntimeGltfInstance, AvMetadata, Texture2D?, Texture2D?, Type)

```
public LoadedUniGLTFAv(GameObject gameObject, RuntimeGltfInstance gltfInstance, AvMetadata
metadata, Texture2D? fullRender, Texture2D? bustRender, Type importerType)
```

Parameters

`gameObject` GameObject

`gltfInstance` RuntimeGltfInstance

`metadata` AvMetadata

`fullRender` Texture2D?

`bustRender` Texture2D?

`importerType` Type

Fields

GLTFInstance

The glTF avatar.

```
public readonly RuntimeGltfInstance GLTFInstance
```

Field Value

RuntimeGltfInstance

Properties

ChannelNames

Case-sensitive keys for all available blend weights.

```
public IReadOnlyCollection<string> ChannelNames { get; }
```

Property Value

IReadOnlyCollection<string>

HasWeights

Does this avatar have any blendshape weights?

```
public bool HasWeights { get; }
```

Property Value

bool

Methods

GetWeight(string)

Gets the weight for the given name.

```
public float GetWeight(string name)
```

Parameters

`name` string

Returns

float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this gets the maximum weight.

HasWeight(string)

Checks if a weight channel with the given name exists.

```
public bool HasWeight(string name)
```

Parameters

`name` string

Returns

bool

ImporterSpecificDispose()

```
protected override void ImporterSpecificDispose()
```

SetWeight(string, float)

Sets the weight for the given name.

```
public void SetWeight(string name, float weight)
```

Parameters

`name` string

`weight` float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this sets the weight for all of them.

TryGetAvatarMaterials()

Tries to get all materials of the avatar.

```
public override IReadOnlyList<Material>? TryGetAvatarMaterials()
```

Returns

IReadOnlyList<Material>

TryGetAvatarMeshes()

Tries to get all meshes of the avatar.

```
public override IReadOnlyList<Mesh>? TryGetAvatarMeshes()
```

Returns

IReadOnlyList<Mesh>

TryGetAvatarRenderers()

Tries to get all renderers of the avatar.

```
public override IReadOnlyList<Renderer>? TryGetAvatarRenderers()
```

Returns

IReadOnlyList<Renderer>

Class LoadedUniVRM10Av

Namespace: Uralstech.AvLoader.Importers

A loaded UniVRM VRM10 avatar.

```
public class LoadedUniVRM10Av : LoadedAv, IImportedAnimator, ILookAtProvider,
IAvatarExpressionProvider, IBlendShapeProviderBulk, IBlendShapeProvider, ICapability
```

Inheritance

object ← LoadedAv ← LoadedUniVRM10Av

Implements

IImportedAnimator, ILookAtProvider, IAvatarExpressionProvider, IBlendShapeProviderBulk, IBlendShapeProvider, ICapability

Inherited Members

LoadedAv.GameObject , LoadedAv.Metadata , LoadedAv.FullRender , LoadedAv.BustRender , LoadedAv.DestroyOnDispose , LoadedAv.LifetimeObjects , LoadedAv.ImporterType , LoadedAv.RegisterLifetimeObject(UnityEngine.Object) , LoadedAv.DeregisterLifetimeObject(UnityEngine.Object) , LoadedAv.TryGetCapability<T>(out T) , LoadedAv.GetCapability<T>() , LoadedAv.GetAvatarMaterials(bool) , LoadedAv.GetAvatarRenderers(bool) , LoadedAv.GetAvatarMeshes(bool) , LoadedAv.ThrowIfDisposed() , LoadedAv.Dispose()

Extension Methods

BlendShapeProviderExtensions.GetWeights(IBlendShapeProvider, ReadOnlySpan<string>, Span<float>) , BlendShapeProviderExtensions.HasAnyWeight(IBlendShapeProvider, string[], out string?) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, IReadOnlyDictionary<string, float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<string>, ReadOnlySpan<float>) , BlendShapeProviderExtensions.SetWeights(IBlendShapeProvider, ReadOnlySpan<(string name, float weight)>)

Constructors

LoadedUniVRM10Av(GameObject, Vrm10Instance, AvMetadata, Texture2D?, Texture2D?, Type)

```
public LoadedUniVRM10Av(GameObject gameObject, Vrm10Instance vrmInstance, AvMetadata
metadata, Texture2D? fullRender, Texture2D? bustRender, Type importerType)
```

Parameters

`gameObject` GameObject

`vrmInstance` Vrm10Instance

`metadata` AvMetadata

`fullRender` Texture2D?

`bustRender` Texture2D?

`importerType` Type

Fields

GLTFInstance

The RuntimeGltfInstance component of the VRM avatar, if it exists.

```
public readonly RuntimeGltfInstance? GLTFInstance
```

Field Value

RuntimeGltfInstance?

VRMInstance

The VRM avatar.

```
public readonly Vrm10Instance VRMInstance
```

Field Value

Vrm10Instance

Properties

Animator

The importer-generated animator.

```
public Animator Animator { get; }
```

Property Value

Animator

ChannelNames

Case-sensitive keys for all available blend weights.

```
public IReadOnlyCollection<string> ChannelNames { get; }
```

Property Value

IReadOnlyCollection<string>

HasWeights

Does this avatar have any blendshape weights?

```
public bool HasWeights { get; }
```

Property Value

bool

Methods

ClearTarget()

Clears the look-at target.

```
public void ClearTarget()
```

Remarks

Implementations should reset the avatar to its default gaze.

GetWeight(string)

Gets the weight for the given name.

```
public float GetWeight(string name)
```

Parameters

`name` string

Returns

float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this gets the maximum weight.

GetWeights(ReadOnlySpan<string>, Span<float>)

Retrieves the weights of the specified blendshape channels into the provided buffer.

```
public void GetWeights(ReadOnlySpan<string> names, Span<float> weights)
```

Parameters

`names` ReadOnlySpan<string>

`weights` Span<float>

Remarks

For channel names that do not exist, the corresponding output value is set to zero.

HasWeight(string)

Checks if a weight channel with the given name exists.

```
public bool HasWeight(string name)
```

Parameters

`name` string

Returns

bool

ImporterSpecificDispose()

```
protected override void ImporterSpecificDispose()
```

SetTarget(Transform)

Sets a transform for the avatar to look at.

```
public void SetTarget(Transform transform)
```

Parameters

`transform` Transform

Remarks

Implementations should track the transform over time.

SetTarget(Vector3)

Sets a fixed world-space position for the avatar to look at.

```
public void SetTarget(Vector3 worldPosition)
```

Parameters

`worldPosition` Vector3

SetWeight(string, float)

Sets the weight for the given name.

```
public void SetWeight(string name, float weight)
```

Parameters

`name` string

`weight` float

Remarks

If multiple blend shape sources in the avatar have the same channel name, this sets the weight for all of them.

SetWeights(ReadOnlySpan<string>, ReadOnlySpan<float>)

Sets the weights of the specified blendshape channels.

```
public int SetWeights(ReadOnlySpan<string> names, ReadOnlySpan<float> weights)
```

Parameters

`names` ReadOnlySpan<string>

`weights` ReadOnlySpan<float>

Returns

int

The number of weights successfully applied.

Remarks

Channel names that do not exist are ignored; the return value indicates how many weights were applied.

SetWeights(ReadOnlySpan<(string name, float weight)>)

Sets multiple blendshape weights using name–value pairs.

```
public int SetWeights(ReadOnlySpan<(string name, float weight)> values)
```

Parameters

`values` ReadOnlySpan<(string name, float weight)>

Returns

int

The number of weights successfully applied.

Remarks

Channel names that do not exist are ignored; the return value indicates how many weights were applied.

TryGetAvatarMaterials()

Tries to get all materials of the avatar.

```
public override IReadOnlyList<Material>? TryGetAvatarMaterials()
```

Returns

IReadOnlyList<Material>

TryGetAvatarMeshes()

Tries to get all meshes of the avatar.

```
public override IReadOnlyList<Mesh>? TryGetAvatarMeshes()
```

Returns

IReadOnlyList<Mesh>

TryGetAvatarRenderers()

Tries to get all renderers of the avatar.

```
public override IReadOnlyList<Renderer>? TryGetAvatarRenderers()
```

Returns

IReadOnlyList<Renderer>

Class UniGLTFAvImporter

Namespace: Uralstech.AvLoader.Importers

Imports glTF avatars using UniGLTF.

```
public class UniGLTFAvImporter : IAvImporter
```

Inheritance

object ← UniGLTFAvImporter

Implements

IAvImporter

Fields

EnableUpdateWhenOffscreen

Should the avatar's SkinnedMeshRenderers be updated when off-screen?

```
public bool EnableUpdateWhenOffscreen
```

Field Value

bool

ImporterContextSettings

Optional importer settings.

```
public ImporterContextSettings? ImporterContextSettings
```

Field Value

ImporterContextSettings?

MaterialGenerator

Optional material generator.

```
public IMaterialDescriptorGenerator? MaterialGenerator
```

Field Value

IMaterialDescriptorGenerator?

ShowMeshes

Should all mesh renderers be enabled on load? (default: true)

```
public bool ShowMeshes
```

Field Value

bool

TextureDeserializer

Optional texture deserialization.

```
public ITextureDeserializer? TextureDeserializer
```

Field Value

ITextureDeserializer?

Methods

ImportAvatarAsync(AvSourceData, bool, CancellationToken)

Tries to import the avatar into a scene as a disabled GameObject, along with metadata and any renders.

```
public Awaitable<LoadedAv?> ImportAvatarAsync(AvSourceData rawData, bool throwOnFail,
CancellationToken token = default)
```

Parameters

`rawData` AvSourceData

The raw avatar data to process.

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar if successful; null on failure.

Remarks

The returned LoadedAv may contain code to handle requirements of format-supporting plugins. For example, GLTFastAvImporter depends on glTFast, which requires that GltfImport object used to import the avatar be active alongside the avatar's GameObject, and should be disposed after the avatar is no longer needed. Thus, LoadedAv implements System.IDisposable.

SupportsFormat(AvModelFileExtension)

Returns true if this importer can handle the given format; false otherwise.

```
public bool SupportsFormat(AvModelFileExtension format)
```

Parameters

`format` AvModelFileExtension

Returns

bool

Class UniVRM10AvImporter

Namespace: Uralstech.AvLoader.Importers

Imports VRM avatars using UniVRM10.

```
public class UniVRM10AvImporter : IAvImporter
```

Inheritance

object ← UniVRM10AvImporter

Implements

IAvImporter

Fields

CanLoadVrm0X

If true, VRM-0.x models are converted to VRM-1.0 and loaded. The components attached to the VRM-0.x model are different. (default: true)

```
public bool CanLoadVrm0X
```

Field Value

bool

ControlRigGenerationOption

ControlRig generation settings. (default: ControlRigGenerationOption.Generate)

```
public ControlRigGenerationOption ControlRigGenerationOption
```

Field Value

ControlRigGenerationOption

ImporterContextSettings

Optional importer settings.

```
public ImporterContextSettings? ImporterContextSettings
```

Field Value

ImporterContextSettings?

MaterialGenerator

Optional material generator.

```
public IMaterialDescriptorGenerator? MaterialGenerator
```

Field Value

IMaterialDescriptorGenerator?

ShowMeshes

Should all mesh renderers be enabled on load? (default: true)

```
public bool ShowMeshes
```

Field Value

bool

SpringboneRuntime

Optional runtime selection for SpringBones.

```
public IVrm10SpringBoneRuntime? SpringboneRuntime
```

Field Value

IVrm10SpringBoneRuntime?

TextureDeserializer

Optional texture deserialization.

```
public ITextureDeserializer? TextureDeserializer
```

Field Value

ITextureDeserializer?

UpdateType

Update type of the loaded avatar.

```
public Vrm10Instance.UpdateTypes UpdateType
```

Field Value

UpdateTypes

VRMMetaInformationCallback

Optional callback called on VRM-0x to VRM-1.0 migration.

```
public Vrm10.VrmMetaInformationCallback? VRMMetaInformationCallback
```

Field Value

Vrm10.VrmMetaInformationCallback?

Methods

ImportAvatarAsync(AvSourceData, bool, CancellationToken)

Tries to import the avatar into a scene as a disabled GameObject, along with metadata and any renders.

```
public Awaitable<LoadedAv?> ImportAvatarAsync(AvSourceData rawData, bool throwOnFail,
CancellationToken token = default)
```

Parameters

`rawData` AvSourceData

The raw avatar data to process.

`throwOnFail` bool

Should this method throw errors on failures or log them as warnings and return null?

`token` CancellationToken

Returns

Awaitable<LoadedAv>

The loaded avatar if successful; null on failure.

Remarks

The returned LoadedAv may contain code to handle requirements of format-supporting plugins. For example, GLTFastAvImporter depends on glTFast, which requires that GltfImport object used to import the avatar be active alongside the avatar's GameObject, and should be disposed after the avatar is no longer needed. Thus, LoadedAv implements System.IDisposable.

SupportsFormat(AvModelFileExtension)

Returns true if this importer can handle the given format; false otherwise.

```
public bool SupportsFormat(AvModelFileExtension format)
```

Parameters

`format` AvModelFileExtension

Returns

bool

Namespace Uralstech.AvLoader.PostProcessors

Classes

ActionPostProcessor

QoL post-processor that runs an Action.

AnimatorPostProcessor

Post-processing step which adds or updates an animation controller in the loaded avatar. Requries Unity's animation module to be enabled.

AsyncFuncPostProcessor

QoL post-processor that runs an async Func.

AvPost

Fluent helper methods for creating common avatar post-processors.

CachePostProcessor

Post-processing step that saves the loaded avatar's data to a local directory for future use. Skips execution if the avatar was originally loaded using FileAvDataLoader, unless overridden via IgnoreOriginalLoaderType.

ConditionalPostProcessor

Post-processing step that runs based on the given condition.

MaterialOverrideConfig

Configuration data for MaterialOverridePostProcessor.

MaterialOverridePostProcessor

Post-processing step that recreates avatar materials using alternative shaders and copies selected property and keyword values from the original materials.

ShaderSwapConfig

Configuration data for ShaderSwapPostProcessor.

ShaderSwapPostProcessor

Post-processing step that swaps the avatar's shaders with the provided set of shaders.

Interfaces

IAvatarDependentComponent

Marks a Component as semantically dependent on a LoadedAv instance.

Class ActionPostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

QoL post-processor that runs an Action.

```
public class ActionPostProcessor : IAvPostProcessor
```

Inheritance

object ← ActionPostProcessor

Implements

IAvPostProcessor

Constructors

ActionPostProcessor(Action<LoadedAv, AvSourceData>)

```
public ActionPostProcessor(Action<LoadedAv, AvSourceData> action)
```

Parameters

`action` Action<LoadedAv, AvSourceData>

Fields

Action

The action to execute.

```
public Action<LoadedAv, AvSourceData> Action
```

Field Value

Action<LoadedAv, AvSourceData>

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
public void PostProcess(LoadedAv avatar, AvSourceData rawData)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

Class AnimatorPostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

Post-processing step which adds or updates an animation controller in the loaded avatar. Requries Unity's animation module to be enabled.

```
public class AnimatorPostProcessor : IAvPostProcessor
```

Inheritance

object ← AnimatorPostProcessor

Implements

IAvPostProcessor

Fields

AnimatorController

Runtime animator controller to assign to the loaded avatar.

```
public RuntimeAnimatorController? AnimatorController
```

Field Value

RuntimeAnimatorController?

AvGenderToAvatarLookup

Lookup table for when you need to assign animation avatars based on the avatar's gender. If not found here, falls back to Avatar.

```
public IReadOnlyDictionary<AvGender, Avatar>? AvGenderToAvatarLookup
```

Field Value

IReadOnlyDictionary<AvGender, Avatar>

Remarks

If the avatar already has an animator, this won't be assigned unless OverrideAvatar is true.

Avatar

Animation avatar to assign to the loaded avatar.

```
public Avatar? Avatar
```

Field Value

Avatar?

Remarks

If the avatar already has an animator, this won't be assigned unless OverrideAvatar is true.

OverrideAvatar

If the avatar already has an animation avatar assigned, should it be overridden?

```
public bool OverrideAvatar
```

Field Value

bool

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
public void PostProcess(LoadedAv avatar, AvSourceData _)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

_ AvSourceData

Class AsyncFuncPostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

QoL post-processor that runs an async Func.

```
public class AsyncFuncPostProcessor : IAsyncAvPostProcessor
```

Inheritance

object ← AsyncFuncPostProcessor

Implements

IAsyncAvPostProcessor

Constructors

AsyncFuncPostProcessor(Func<LoadedAv, AvSourceData, CancellationToken, Awaitable>)

```
public AsyncFuncPostProcessor(Func<LoadedAv, AvSourceData, CancellationToken,
Awaitable> func)
```

Parameters

`func` Func<LoadedAv, AvSourceData, CancellationToken, Awaitable>

Fields

Func

The System.Func<T1, T2, T3, TResult> to execute.

```
public Func<LoadedAv, AvSourceData, CancellationToken, Awaitable> Func
```

Field Value

Func<LoadedAv, AvSourceData, CancellationToken, Awaitable>

Methods

PostProcessAsync(LoadedAv, AvSourceData, CancellationToken)

Runs a post-processing step on the given avatar.

```
public Awaitable PostProcessAsync(LoadedAv avatar, AvSourceData rawData, CancellationToken
token = default)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

`token` CancellationToken

Returns

Awaitable

Class AvPost

Namespace: Uralstech.AvLoader.PostProcessors

Fluent helper methods for creating common avatar post-processors.

```
public static class AvPost
```

Inheritance

object ← AvPost

Methods

CacheTo(string, bool, AvImageFileExtension, AvImageFileExtension, int, int, bool)

Creates a post-processor that caches the loaded avatar's source data to disk.

```
public static CachePostProcessor CacheTo(string directory, bool useAvatarIdAsChildDirName =
true, AvImageFileExtension fullRenderImageFormat = AvImageFileExtension.JPG,
AvImageFileExtension bustRenderImageFormat = AvImageFileExtension.JPG, int
fullRenderJPEGQuality = 100, int bustRenderJPEGQuality = 100, bool ignoreOriginalLoaderType
= false)
```

Parameters

`directory` string

The base directory to save the cached avatar data to.

`useAvatarIdAsChildDirName` bool

If true, the avatar is stored in a child directory named after its ID.

`fullRenderImageFormat` AvImageFileExtension

The image format to use when saving the full-body render.

`bustRenderImageFormat` AvImageFileExtension

The image format to use when saving the bust render.

`fullRenderJPEGQuality` int

JPEG quality to use when saving the full-body render.

`bustRenderJPEGQuality` int

JPEG quality to use when saving the bust render.

`ignoreOriginalLoaderType` bool

If true, caching will occur even if the avatar was originally loaded from disk.

Returns

CachePostProcessor

Remarks

This only works for formats where the model does not depend on resources stored separately from the source file (Model). For example, avatars loaded from OBJ files with external texture files will not work.

ConfigureAnimator(RuntimeAnimatorController?, Avatar?, IReadOnlyDictionary<AvGender, Avatar>?, bool)

Creates a post-processor that configures an Animator on the loaded avatar.

```
public static AnimatorPostProcessor ConfigureAnimator(RuntimeAnimatorController?
animatorController = null, Avatar? avatar = null, IReadOnlyDictionary<AvGender, Avatar>?
avGenderToAvatarLookup = null, bool overrideAvatar = false)
```

Parameters

`animatorController` RuntimeAnimatorController?

The runtime animator controller to assign to the avatar.

`avatar` Avatar?

The animation avatar to assign if no gender-specific match is found.

`avGenderToAvatarLookup` IReadOnlyDictionary<AvGender, Avatar>

Optional lookup table mapping avatar gender to animation avatars.

`overrideAvatar` bool

If true, any existing animation avatar will be overridden.

Returns

AnimatorPostProcessor

Do(Action<LoadedAv, AvSourceData>)

Creates a post-processor that executes the given action.

```
public static ActionPostProcessor Do(Action<LoadedAv, AvSourceData> action)
```

Parameters

`action` Action<LoadedAv, AvSourceData>

The action to execute during post-processing.

Returns

ActionPostProcessor

DoAsync(Func<LoadedAv, AvSourceData, CancellationToken, Awaitable>)

Creates an asynchronous post-processor that executes the given function.

```
public static AsyncFuncPostProcessor DoAsync(Func<LoadedAv, AvSourceData, CancellationToken,
Awaitable> func)
```

Parameters

`func` Func<LoadedAv, AvSourceData, CancellationToken, Awaitable>

The async function to execute during post-processing.

Returns

AsyncFuncPostProcessor

EnsureComponent<T>()

Creates a post-processor that will ensure the loaded avatar has a component of type `T`.

```
public static ActionPostProcessor EnsureComponent<T>() where T : Component
```

Returns

ActionPostProcessor

Type Parameters

`T`

The component type.

EnsureDependentComponent<T>()

Creates a post-processor that ensures the avatar has a component of type `T` and attaches it as dependent on the loaded avatar.

```
public static ActionPostProcessor EnsureDependentComponent<T>() where T :
Component, IAvatarDependentComponent
```

Returns

ActionPostProcessor

Type Parameters

`T`

The dependent component type.

OverrideMaterials(MaterialOverrideConfig)

Creates a post-processor that recreates avatar materials using alternative shaders and applies configured property and keyword overrides.

```
public static MaterialOverridePostProcessor OverrideMaterials(MaterialOverrideConfig config)
```

Parameters

`config` MaterialOverrideConfig

Configuration that defines material override rules, including source shaders, target shaders, and property and keyword mappings.

Returns

MaterialOverridePostProcessor

SwapShaders(ShaderSwapConfig)

Creates a post-processor that swaps shaders of the loaded avatar based on the given `config`.

```
public static ShaderSwapPostProcessor SwapShaders(ShaderSwapConfig config)
```

Parameters

`config` ShaderSwapConfig

Configuration that includes a map of shaders to be replaced and their replacements.

Returns

ShaderSwapPostProcessor

When(Func<LoadedAv, AvSourceData, bool>, IAsyncAvPostProcessor)

Creates a conditional asynchronous post-processor that runs only when the given condition is met.

```
public static ConditionalPostProcessor When(Func<LoadedAv, AvSourceData, bool> condition,
IAsyncAvPostProcessor onCondition)
```

Parameters

`condition` Func<LoadedAv, AvSourceData, bool>

A predicate determining whether the post-processor should run.

`onCondition` IAsyncAvPostProcessor

The async post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

When(Func<LoadedAv, AvSourceData, bool>, IAvPostProcessor)

Creates a conditional post-processor that runs only when the given condition is met.

```
public static ConditionalPostProcessor When(Func<LoadedAv, AvSourceData, bool> condition,
IAvPostProcessor onCondition)
```

Parameters

`condition` Func<LoadedAv, AvSourceData, bool>

A predicate determining whether the post-processor should run.

`onCondition` IAvPostProcessor

The post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

WhenImportedWith<T>(IAsyncAvPostProcessor)

Creates a conditional asynchronous post-processor that runs only when the avatar is imported using an importer of type `T`.

```
public static ConditionalPostProcessor WhenImportedWith<T>(IAsyncAvPostProcessor
onCondition) where T : IAvImporter
```

Parameters

`onCondition` IAsyncAvPostProcessor

The post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

Type Parameters

`T`

The importer type that triggers execution of `onCondition`.

WhenImportedWith<T>(IAvPostProcessor)

Creates a conditional post-processor that runs only when the avatar is imported using an importer of type `T`.

```
public static ConditionalPostProcessor WhenImportedWith<T>(IAvPostProcessor onCondition)
where T : IAvImporter
```

Parameters

`onCondition` IAvPostProcessor

The post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

Type Parameters

`T`

The importer type that triggers execution of `onCondition`.

WhenLoadedWith<T>(IAsyncAvPostProcessor)

Creates a conditional asynchronous post-processor that runs only when the avatar data is loaded using a loader of type `T`.

```
public static ConditionalPostProcessor WhenLoadedWith<T>(IAsyncAvPostProcessor onCondition)
where T : IAvDataLoader
```

Parameters

`onCondition` IAsyncAvPostProcessor

The post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

Type Parameters

`T`

The loader type that triggers execution of `onCondition`.

WhenLoadedWith<T>(IAvPostProcessor)

Creates a conditional post-processor that runs only when the avatar data is loaded using a loader of type `T`.

```
public static ConditionalPostProcessor WhenLoadedWith<T>(IAvPostProcessor onCondition) where
T : IAvDataLoader
```

Parameters

`onCondition` IAvPostProcessor

The post-processor to execute when the condition is met.

Returns

ConditionalPostProcessor

Type Parameters

`T`

The loader type that triggers execution of `onCondition`.

Class CachePostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

Post-processing step that saves the loaded avatar's data to a local directory for future use. Skips execution if the avatar was originally loaded using FileAvDataLoader, unless overridden via IgnoreOriginalLoaderType.

```
public class CachePostProcessor : IAsyncAvPostProcessor
```

Inheritance

object ← CachePostProcessor

Implements

IAsyncAvPostProcessor

Remarks

This only works for formats where the model does not depend on resources stored separately from the source file (Model). For example, avatars loaded from OBJ files with external texture files will not work.

Constructors

CachePostProcessor(string, bool)

```
public CachePostProcessor(string baseDirectory, bool useAvatarIdAsChildDirName = true)
```

Parameters

`baseDirectory` string

`useAvatarIdAsChildDirName` bool

Fields

BaseDirectory

The base directory to save the avatar to.

```
public string BaseDirectory
```

Field Value

string

Remarks

If UseAvatarIdAsDirName is true (default), the avatar will be stored in a new directory at "Base Directory/Id".

BustRenderJPEGQuality

The quality to encode the avatar's bust render, if saving as JPEG.

```
public int BustRenderJPEGQuality
```

Field Value

int

FullRenderJPEGQuality

The quality to encode the avatar's full render, if saving as JPEG.

```
public int FullRenderJPEGQuality
```

Field Value

int

IgnoreOriginalLoaderType

Should the avatar be saved even if it was loaded from the local filesystem?

```
public bool IgnoreOriginalLoaderType
```

Field Value

bool

UseAvatarIdAsDirName

If true (default), the avatar will be stored in a new directory at "BaseDirectory/Id". Otherwise, Base Directory is used as-is.

```
public bool UseAvatarIdAsDirName
```

Field Value

bool

Properties

BustRenderImageFormat

The file extension and format for saving the avatar's bust render. Cannot be presumptive, like JPEGAny.

```
public AvImageFileExtension BustRenderImageFormat { get; set; }
```

Property Value

AvImageFileExtension

FullRenderImageFormat

The file extension and format for saving the avatar's full render. Cannot be presumptive, like JPEGAny.

```
public AvImageFileExtension FullRenderImageFormat { get; set; }
```

Property Value

AvImageFileExtension

Methods

PostProcessAsync(LoadedAv, AvSourceData, CancellationToken)

Runs a post-processing step on the given avatar.

```
public Awaitable PostProcessAsync(LoadedAv avatar, AvSourceData rawData, CancellationToken
token = default)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

`token` CancellationToken

Returns

Awaitable

Class ConditionalPostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

Post-processing step that runs based on the given condition.

```
public class ConditionalPostProcessor : IAvPostProcessor, IAsyncAvPostProcessor
```

Inheritance

object ← ConditionalPostProcessor

Implements

IAvPostProcessor, IAsyncAvPostProcessor

Constructors

ConditionalPostProcessor(Func<LoadedAv, AvSourceData, bool>, IAsyncAvPostProcessor)

```
public ConditionalPostProcessor(Func<LoadedAv, AvSourceData, bool> condition,
IAsyncAvPostProcessor asyncPostProcessor)
```

Parameters

`condition` Func<LoadedAv, AvSourceData, bool>

`asyncPostProcessor` IAsyncAvPostProcessor

ConditionalPostProcessor(Func<LoadedAv, AvSourceData, bool>, IAvPostProcessor)

```
public ConditionalPostProcessor(Func<LoadedAv, AvSourceData, bool> condition,
IAvPostProcessor postProcessor)
```

Parameters

`condition` Func<LoadedAv, AvSourceData, bool>

`postProcessor` IAvPostProcessor

Fields

Condition

Condition callback.

```
public Func<LoadedAv, AvSourceData, bool> Condition
```

Field Value

Func<LoadedAv, AvSourceData, bool>

Properties

PostProcessor

Post-processor to run if Condition is met, can be of any type inheriting from IAvPostProcessor or IAsyncAvPostProcessor.

```
public object PostProcessor { get; set; }
```

Property Value

object

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
public void PostProcess(LoadedAv avatar, AvSourceData rawData)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

PostProcessAsync(LoadedAv, AvSourceData, CancellationToken)

Runs a post-processing step on the given avatar.

```
public Awaitable PostProcessAsync(LoadedAv avatar, AvSourceData rawData, CancellationToken
token = default)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

`token` CancellationToken

Returns

Awaitable

Interface IAvatarDependentComponent

Namespace: Uralstech.AvLoader.PostProcessors

Marks a Component as semantically dependent on a LoadedAv instance.

```
public interface IAvatarDependentComponent
```

Methods

OnAvatarAttached(LoadedAv)

Called when the component is attached to an avatar via a dependent-component post-processor.

```
void OnAvatarAttached(LoadedAv avatar)
```

Parameters

`avatar` LoadedAv

The loaded avatar this component depends on.

Class MaterialOverrideConfig

Namespace: Uralstech.AvLoader.PostProcessors

Configuration data for MaterialOverridePostProcessor.

```
public class MaterialOverrideConfig : ScriptableObject, ISerializationCallbackReceiver
```

Inheritance

object ← MaterialOverrideConfig

Implements

ISerializationCallbackReceiver

Fields

MaterialOverrides

Runtime map of source shaders to their material override definitions.

```
public Dictionary<Shader, RuntimeMaterialOverrideDefinition> MaterialOverrides
```

Field Value

Dictionary<Shader, RuntimeMaterialOverrideDefinition>

Class MaterialOverridePostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

Post-processing step that recreates avatar materials using alternative shaders and copies selected property and keyword values from the original materials.

```
public class MaterialOverridePostProcessor : IAvPostProcessor
```

Inheritance

object ← MaterialOverridePostProcessor

Implements

IAvPostProcessor

Constructors

MaterialOverridePostProcessor(MaterialOverrideConfig)

```
public MaterialOverridePostProcessor(MaterialOverrideConfig configuration)
```

Parameters

`configuration` MaterialOverrideConfig

Fields

Configuration

Configuration which defines the material override rules.

```
public MaterialOverrideConfig Configuration
```

Field Value

MaterialOverrideConfig

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
public void PostProcess(LoadedAv avatar, AvSourceData rawData)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`rawData` AvSourceData

The raw data of the loaded avatar.

Class ShaderSwapConfig

Namespace: Uralstech.AvLoader.PostProcessors

Configuration data for ShaderSwapPostProcessor.

```
public class ShaderSwapConfig : ScriptableObject, ISerializationCallbackReceiver
```

Inheritance

object ← ShaderSwapConfig

Implements

ISerializationCallbackReceiver

Fields

FallbackShader

Optional shader used when no entry exists in ShaderMap.

```
public Shader? FallbackShader
```

Field Value

Shader?

ShaderMap

Maps source shaders to their replacement shaders.

```
public Dictionary<Shader, Shader> ShaderMap
```

Field Value

Dictionary<Shader, Shader>

Class ShaderSwapPostProcessor

Namespace: Uralstech.AvLoader.PostProcessors

Post-processing step that swaps the avatar's shaders with the provided set of shaders.

```
public class ShaderSwapPostProcessor : IAvPostProcessor
```

Inheritance

object ← ShaderSwapPostProcessor

Implements

IAvPostProcessor

Constructors

ShaderSwapPostProcessor(ShaderSwapConfig)

```
public ShaderSwapPostProcessor(ShaderSwapConfig configuration)
```

Parameters

`configuration` ShaderSwapConfig

Fields

Configuration

Configuration which defines the shaders to swap.

```
public ShaderSwapConfig Configuration
```

Field Value

ShaderSwapConfig

Methods

PostProcess(LoadedAv, AvSourceData)

Runs a post-processing step on the given avatar.

```
public void PostProcess(LoadedAv avatar, AvSourceData _)
```

Parameters

`avatar` LoadedAv

The loaded avatar to process.

`_` AvSourceData

Namespace Uralstech.AvLoader.PostProcessors.Rendering

Classes

RuntimeMaterialOverrideDefinition
: Defines how a material using a specific shader is recreated using another shader, with selected properties and keywords copied.

ShaderKeywordMapping
: Describes how a single shader keyword is copied from a source material to a target material.

ShaderKeywordRule
: Defines a conditional rule that enables or disables a target shader keyword based on the state of a source keyword.

ShaderMapping
: Defines a mapping from a source shader to a target shader.

ShaderPropertyMapping
: Describes how a single shader property value is copied from a source material to a target material.

Enums

ShaderPropertyType
: Represents the supported shader property data types that can be transferred between materials, based on the Material `.GetX` and `.SetX` methods.

Class RuntimeMaterialOverrideDefinition

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Defines how a material using a specific shader is recreated using another shader, with selected properties and keywords copied.

```
public class RuntimeMaterialOverrideDefinition
```

Inheritance

object ← RuntimeMaterialOverrideDefinition

Constructors

RuntimeMaterialOverrideDefinition(Shader, IReadOnlyList<ShaderPropertyMapping>, IReadOnlyList<ShaderKeywordMapping>?, IReadOnlyList<ShaderKeywordRule>?)

```
public RuntimeMaterialOverrideDefinition(Shader target, IReadOnlyList<ShaderPropertyMapping>
propertyMappings, IReadOnlyList<ShaderKeywordMapping>? keywordMappings = null,
IReadOnlyList<ShaderKeywordRule>? keywordRules = null)
```

Parameters

`target` Shader

`propertyMappings` IReadOnlyList<ShaderPropertyMapping>

`keywordMappings` IReadOnlyList<ShaderKeywordMapping>

`keywordRules` IReadOnlyList<ShaderKeywordRule>

Fields

Target

The shader used by the newly created material.

```
public Shader Target
```

Field Value

Shader

Properties

KeywordMappings

Optional keyword mappings that directly copy enabled/disabled states from source keywords to target keywords.

```
public IReadOnlyList<ShaderKeywordMapping>? KeywordMappings { get; set; }
```

Property Value

IReadOnlyList<ShaderKeywordMapping>

Remarks

For conditional or overridden behavior, use KeywordRules instead.

Exceptions

ArgumentException

Thrown if any mapping is invalid or if duplicate source or target keywords are found.

KeywordRules

Optional rules that conditionally enable or disable shader keywords on the new material.

```
public IReadOnlyList<ShaderKeywordRule>? KeywordRules { get; set; }
```

Property Value

IReadOnlyList<ShaderKeywordRule>

Exceptions

ArgumentException

Thrown if any rule has an empty or null source or target keyword.

PropertyMappings

Property mappings used to copy values from the source material to the newly created material.

```
public IReadOnlyList<ShaderPropertyMapping> PropertyMappings { get; set; }
```

Property Value

IReadOnlyList<ShaderPropertyMapping>

Exceptions

ArgumentException

Thrown if any mapping is invalid or if duplicate source or target properties are found.

Class ShaderKeywordMapping

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Describes how a single shader keyword is copied from a source material to a target material.

```
[Serializable]
public class ShaderKeywordMapping
```

Inheritance

object ← ShaderKeywordMapping

Remarks

Use ShaderKeywordRule for conditional behavior. A mapping directly mirrors the enabled state of Source onto Target.

Fields

Source

The name of the keyword on the source shader.

```
public string? Source
```

Field Value

string

Target

The name of the corresponding keyword on the target shader.

```
public string? Target
```

Field Value

string

Methods

IsValid()

Returns true if both source and target keyword names are non-empty.

```
public bool IsValid()
```

Returns

bool

IsValid(IReadOnlyList<ShaderKeywordMapping>?)

Returns true if all keyword mappings in the list are valid and contain no duplicates.

```
public static bool IsValid(IReadOnlyList<ShaderKeywordMapping>? mappings)
```

Parameters

`mappings` IReadOnlyList<ShaderKeywordMapping>

Returns

bool

Class ShaderKeywordRule

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Defines a conditional rule that enables or disables a target shader keyword based on the state of a source keyword.

```
[Serializable]
public class ShaderKeywordRule
```

Inheritance

object ← ShaderKeywordRule

Remarks

If you want to copy the enabled/disabled state of a keyword to another one, use ShaderKeyword Mapping instead.

Fields

Source

The source shader keyword to test.

```
public string? Source
```

Field Value

string

SourceRequiredState

The required enabled state of the source keyword for this rule to apply.

```
public bool SourceRequiredState
```

Field Value

bool

Target

The target shader keyword to modify.

```
public string? Target
```

Field Value

string

TargetResultState

The keyword state applied to the target material when the rule matches.

```
public bool TargetResultState
```

Field Value

bool

Methods

IsValid()

Returns true if both source and target keyword names are non-empty.

```
public bool IsValid()
```

Returns

bool

IsValid(IReadOnlyList<ShaderKeywordRule>?)

Returns true if all keyword rules in the list are valid.

```
public static bool IsValid(IReadOnlyList<ShaderKeywordRule>? rules)
```

Parameters

`rules` IReadOnlyList<ShaderKeywordRule>

Returns

bool

Class ShaderMapping

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Defines a mapping from a source shader to a target shader.

```
[Serializable]
public class ShaderMapping
```

Inheritance

object ← ShaderMapping

Fields

Source

The original shader to be replaced.

```
public Shader? Source
```

Field Value

Shader?

Target

The shader that replaces the source shader.

```
public Shader? Target
```

Field Value

Shader?

Methods

IsValid()

Returns true if both source and target shaders are assigned.

```
public bool IsValid()
```

Returns

bool

Class ShaderPropertyMapping

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Describes how a single shader property value is copied from a source material to a target material.

```
[Serializable]
public class ShaderPropertyMapping
```

Inheritance

object ← ShaderPropertyMapping

Fields

Source

The name of the property on the source shader.

```
public string? Source
```

Field Value

string

Target

The name of the corresponding property on the target shader.

```
public string? Target
```

Field Value

string

Type

The expected data type of the shader property.

```
public ShaderPropertyType Type
```

Field Value

ShaderPropertyType

Methods

IsValid()

Returns true if both source and target property names are non-empty.

```
public bool IsValid()
```

Returns

bool

Enum ShaderPropertyType

Namespace: Uralstech.AvLoader.PostProcessors.Rendering

Represents the supported shader property data types that can be transferred between materials, based on the Material `.GetX` and `.SetX` methods.

```
public enum ShaderPropertyType
```

Fields

`Color = 2`

`Float = 0`

`Inverse01RangeFloat = 5`

Inverts the float value in 0-1 range before assignment.

E.g. assigning a "smoothness" factor to a shader that expects a "roughness" factor.

`Matrix = 4`

`Texture = 3`

`Vector = 1`

Namespace Uralstech.AvLoader.Utils

Classes

ListExtensions

Utility extensions for lists.

TextureExtensions

Utility extensions for texture-related operations.

UnityWebRequestException

Exception thrown when a UnityWebRequest fails, providing detailed request information.

WebRequestExtensions

Utility extensions for web requests.

Class ListExtensions

Namespace: Uralstech.AvLoader.Utils

Utility extensions for lists.

```
public static class ListExtensions
```

Inheritance

object ← ListExtensions

Remarks

This class is public to allow package users to reuse these extensions if useful. However, it should not be considered stable and is not part of the supported public API. It exists solely as an internal utility and may change or be removed at any time.

Methods

CreateCopy<T>(IReadOnlyList<T>)

Creates a shallow copy of the current list.

```
public static T[] CreateCopy<T>(this IReadOnlyList<T> current)
```

Parameters

`current` IReadOnlyList<T>

Returns

T[]

Type Parameters

`T`

Class TextureExtensions

Namespace: Uralstech.AvLoader.Utils

Utility extensions for texture-related operations.

```
public static class TextureExtensions
```

Inheritance

object ← TextureExtensions

Remarks

This class is public to allow package users to reuse these extensions if useful. However, it should not be considered stable and is not part of the supported public API. It exists solely as an internal utility and may change or be removed at any time.

Methods

TryDecodeImage(byte[]?, out Texture2D?, string?, bool)

```
public static bool TryDecodeImage(this byte[]? current, out Texture2D? result, string? name
= null, bool throwOnFail = false)
```

Parameters

`current` byte[]

`result` Texture2D?

`name` string

`throwOnFail` bool

Returns

bool

TryDecodeImage(ReadOnlySpan<byte>, out Texture2D?, string?, bool)

Tries decoding a Texture2D from binary data using ImageConversion.LoadImage(Texture2D, ReadOnlySpan<byte>).

```
public static bool TryDecodeImage(this ReadOnlySpan<byte> current, out Texture2D? result,
string? name = null, bool throwOnFail = false)
```

Parameters

`current` ReadOnlySpan<byte>

`result` Texture2D?

`name` string

Name to give the created texture, and for error logs.

`throwOnFail` bool

Should the method throw if it couldn't load the image data?

Returns

bool

TryGetTextureAsync(UnityWebRequest?, bool, string?, CancellationToken)

Awaits for the DownloadHandlerTexture of the current UnityWebRequest to finish processing its texture.

```
public static Awaitable<Texture2D?> TryGetTextureAsync(this UnityWebRequest? request, bool
throwOnFail = false, string? name = null, CancellationToken token = default)
```

Parameters

`request` UnityWebRequest?

`throwOnFail` bool

Should the method throw if it couldn't load the image data?

`name` string

Name for error logs.

`token` CancellationToken

Returns

Awaitable<Texture2D?>

Class UnityWebRequestException

Namespace: Uralstech.AvLoader.Utils

Exception thrown when a UnityWebRequest fails, providing detailed request information.

```
public sealed class UnityWebRequestException : IOException
```

Inheritance

object ← Exception ← SystemException ← IOException ← UnityWebRequestException

Constructors

UnityWebRequestException(UnityWebRequest)

```
public UnityWebRequestException(UnityWebRequest request)
```

Parameters

`request` UnityWebRequest

Fields

DownloadHandlerError

The error message reported by the DownloadHandler.

```
public readonly string DownloadHandlerError
```

Field Value

string

Error

The error message reported by the UnityWebRequest.

```
public readonly string Error
```

Field Value

string

ResponseCode

The HTTP response code (if any).

```
public readonly long ResponseCode
```

Field Value

long

Url

The URL that failed to load.

```
public readonly string Url
```

Field Value

string

Class WebRequestExtensions

Namespace: Uralstech.AvLoader.Utils

Utility extensions for web requests.

```
public static class WebRequestExtensions
```

Inheritance

object ← WebRequestExtensions

Remarks

This class is public to allow package users to reuse these extensions if useful. However, it should not be considered stable and is not part of the supported public API. It exists solely as an internal utility and may change or be removed at any time.

Methods

AbortAll(List<UnityWebRequest>)

Aborts all UnityWebRequests in the current list.

```
public static void AbortAll(this List<UnityWebRequest> current)
```

Parameters

`current` List<UnityWebRequest>

AsTask(AsyncOperation)

Wraps an AsyncOperation as a System.Threading.Tasks.Task.

```
public static Task AsTask(this AsyncOperation current)
```

Parameters

`current` AsyncOperation

Returns

Task

DisposeAll(List<UnityWebRequest>)

Disposes all UnityWebRequests in the current list.

```
public static void DisposeAll(this List<UnityWebRequest> current)
```

Parameters

`current` List<UnityWebRequest>

GetErred(List<UnityWebRequest>)

Enumerates the erred/failed UnityWebRequests in the current list.

```
public static IEnumerable<UnityWebRequest> GetErred(this List<UnityWebRequest> current)
```

Parameters

`current` List<UnityWebRequest>

Returns

IEnumerable<UnityWebRequest>

SendAll(List<UnityWebRequest>)

Sends all UnityWebRequests in the current list, and creates a System.Threading.Tasks.Task[] for their async operations.

```
public static Task[] SendAll(this List<UnityWebRequest> current)
```

Parameters

`current` List<UnityWebRequest>

Returns

Task[]